Lab 10.1: Adding Print and Print Preview to TextView

In this lab, you will implement the printing override functions and provide a way to calculate printer metrics.

To see a demonstration of the solution for this lab, click this icon.
(CD-ROM plays the demonstration, "Lab 10.1 Demonstration.")

Estimated time to complete this lab: **30 minutes**

To complete the exercises in this lab, you must have the required software. For detailed information about the labs and setup for the labs, see Labs in this course.

The code that forms the starting point for this lab is located in the folder \Labs\Ch10\Lab10.1\Ex01.

The solution code for this lab is located in the folder \Labs\Ch10\Lab10.1\Ex01\Solution.

Objectives

After completing this lab, you will be able to:

- Prepare the **Print** dialog box.
- Display a print preview.
- Respond to the user's selection in the **Print** dialog box.
- Print data from a non-document class.

Prerequisites

Before working on this lab, you should be familiar with the following:

- Member functions of the **CDC** class

You may find it helpful to review the MSDN Library Visual Studio 98 Help topics on graphics before attempting this lab.

Exercises

The following exercise provides practice working with the concepts and techniques covered in this chapter.

Exercise 1: Adding Printing to the Text Viewer

In Lab 4.3 (Optional), Exercise 1: Implementing a Basic Text Viewer, you implemented code to create a multiple document interface (MDI) text viewer with file selection, display, and scrolling capabilities. In this exercise, you will implement the additional support needed to print documents. You will also implement the printing override functions and provide a way to calculate printer metrics.

319 Exercise 1: Adding Printing to the Text Viewer

The code that forms the starting point for this exercise is in \Labs\Ch10\Lab10.1\Ex01.

In Lab 4.3 (Optional), Exercise 1: Implementing a Basic Text Viewer, you implemented code to create a multiple document interface (MDI) text viewer with file selection, display, and scrolling capabilities. In this exercise, you will implement the additional support needed to print documents. You will implement the printing override functions and provide a way to calculate printer metrics.

▶ Prepare the interface file for printing

☐ 1. In the file TextView.h, declare instance variables for the following key printer characteristics:

- Size of a character cell
- Number of printable lines per page
- Number of pages that are to be printed

These declarations are all part of the protected implementation section:

```
//print properties
CSize   m_PrintCharSize;
int     m_nLinesPerPrintPage;
int     m_nPrintPages;
```

☐ 2. Create public accessor methods for lines and pages as shown:

```
int GetLinesPerPrintPage() const
    { return m_nLinesPerPrintPage; }
int GetPrintPageCount() const
    {   return m_nPrintPages; }
```

☐ 3. Save TextView.h.

▶ Modify the constructor to initialize the new instance variables

☐ 1. In the file TextView.cpp, initialize both the lines per page and the number of pages to 0. The complete constructor code is as follows:

```
CTextView::CTextView()
        :   m_ViewCharSize(0,0),
            m_DocSize(0,0)

{
    m_pFont = NULL;
    //print
    m_nLinesPerPrintPage= 0;
    m_nPrintPages       = 0;
}
```

☐ 2. Save TextView.cpp.

Notes

▶ Implement OnPreparePrinting

In this procedure, you will override the **OnPreparePrinting** function to display the **Print** dialog box.

☐ 1. Calculate the printer's metrics as follows:

```
ComputePrintMetrics();
```

☐ 2. Set the number of the last page as follows:

```
pInfo->SetMaxPage(GetPrintPageCount());
```

☐ 3. Disable the **Print To File** button as follows:

```
pInfo->m_pPD->m_pd.Flags |= PD_DISABLEPRINTTOFILE;
```

☐ 4. Display the **Print and Print Preview** dialog box as follows:

```
BOOL bRet = DoPreparePrinting(pInfo);
```

☐ 5. Set the number of preview pages to 1 as follows:

```
pInfo->m_nNumPreviewPages = 1;
```

☐ 6. Return from the dialog box as follows:

```
return (bRet);
```

☐ 7. Save TextView.cpp.

To see an example of how your code should look, click this icon.
(CD-ROM displays the following sample code.)

```
BOOL CTextView::OnPreparePrinting(CPrintInfo* pInfo)
{

    ComputePrintMetrics();
    pInfo->SetMaxPage(GetPrintPageCount());
    pInfo->m_pPD->m_pd.Flags |= PD_DISABLEPRINTTOFILE;

    BOOL bRet = DoPreparePrinting(pInfo);

    pInfo->m_nNumPreviewPages = 1;
    return bRet;
}
```

▶ Implement ComputePrintMetrics

In this procedure, you will implement a function named **ComputePrintMetrics** that calculates the basic metrics for the printer device context (DC).

☐ 1. Declare a protected function in TextView.h as follows:

```
void ComputePrintMetrics();
```

☐ 2. Define the function in TextView.cpp as follows:

```
void CTextView::ComputePrintMetrics()
```

☐ 3. Get a pointer to the document object as follows:

```
CTextDoc* pDoc = GetDocument();
```

Notes

☐ 4. Create the printer DC and set its mapping mode as follows:

```
CDC* pDC = CreatePrinterDC();
pDC->SetMapMode(MM_LOENGLISH);
```

☐ 5. Select the font into the DC, and retrieve its text metrics as follows:

```
CFont* pPreviousFont = pDC->SelectObject(GetFont());
TEXTMETRIC tm;
pDC->GetTextMetrics(&tm);
```

☐ 6. Calculate the size of the character cells in device metrics as follows:

```
m_PrintCharSize.cy = tm.tmHeight + tm.tmExternalLeading;
m_PrintCharSize.cx = tm.tmAveCharWidth;
pDC->LPtoDP(&m_PrintCharSize);
```

☐ 7. Retrieve the height of a page and calculate the number of lines per page as follows:

```
int nPhysDevHeight = pDC->GetDeviceCaps(PHYSICALHEIGHT);
int nPhysOffsetY = pDC->GetDeviceCaps(PHYSICALOFFSETY);
nPhysDevHeight -= nPhysOffsetY * 2;
m_nLinesPerPrintPage = abs(nPhysDevHeight/m_PrintCharSize.cy );
```

☐ 8. Set the margin to 8 lines as follows:

```
m_nLinesPerPrintPage -= 8;
```

☐ 9. From the lines per page, calculate the number of pages that will be printed as follows:

```
m_nPrintPages = pDoc->GetLineList()->GetCount() /
                                        m_nLinesPerPrintPage;
if (pDoc->GetLineList()->GetCount() % m_nLinesPerPrintPage)
{
    m_nPrintPages++;
}
```

☐ 10. Replace the old font in the DC before you destroy the DC as follows:

```
if(pPreviousFont)
{
    pDC->SelectObject(pPreviousFont);
}
::DeleteDC(pDC->GetSafeHdc());
```

☐ 11. Save TextView.cpp.

Notes

To see an example of how your code should look, click this icon.

(CD-ROM displays the following sample code.)

```
void CTextView::ComputePrintMetrics()
{
    CTextDoc* pDoc = GetDocument();

    CDC* pDC = CreatePrinterDC();
    pDC->SetMapMode(MM_LOENGLISH);

    CFont* pPreviousFont = pDC->SelectObject(GetFont());
    TEXTMETRIC tm;
    pDC->GetTextMetrics(&tm);

    m_PrintCharSize.cy = tm.tmHeight + tm.tmExternalLeading;
    m_PrintCharSize.cx = tm.tmAveCharWidth;
    pDC->LPtoDP(&m_PrintCharSize);

    int nPhysDevHeight = pDC->GetDeviceCaps(PHYSICALHEIGHT);
    int nPhysOffsetY = pDC->GetDeviceCaps(PHYSICALOFFSETY);
    nPhysDevHeight -= nPhysOffsetY * 2;

    m_nLinesPerPrintPage = abs(nPhysDevHeight/m_PrintCharSize.cy );
    m_nLinesPerPrintPage -= 8;

    m_nPrintPages = pDoc->GetLineList()->GetCount() /
                                    m_nLinesPerPrintPage;
    if (pDoc->GetLineList()->GetCount() % m_nLinesPerPrintPage)
    {
        m_nPrintPages++;
    }

    if(pPreviousFont)
    {
        pDC->SelectObject(pPreviousFont);
    }
    ::DeleteDC(pDC->GetSafeHdc());
}
```

Notes

▶ **Implement the CreatePrinterDC function**

☐ 1. Declare a protected function in TextView.h as follows:

```
CDC* CreatePrinterDC();
```

☐ 2. Save TextView.h.

☐ 3. In TextView.cpp, define **CreatePrinterDC** as follows:

```
CDC* CTextView::CreatePrinterDC()
{
    PRINTDLG PrtDlg;
    HDC      hDC;
```

☐ 4. Determine whether or not you have a printer installed as follows:

```
if (!AfxGetApp()->GetPrinterDeviceDefaults(&PrtDlg))
```

☐ 5. If you don't have a printer, display text on the screen DC:

```
hDC = ::CreateDC("display",NULL,NULL,NULL);
```

☐ 6. If you do have a printer, show the **Print** dialog box as follows:

```
CPrintDialog dlg(FALSE);
dlg.m_pd.hDevMode = PrtDlg.hDevMode;
dlg.m_pd.hDevNames = PrtDlg.hDevNames;
hDC = dlg.CreatePrinterDC();
```

☐ 7. Create a pointer to a **CDC** instance from the handle to the DC that you have been using as follows:

```
CDC* pDC = CDC::FromHandle(hDC);
```

☐ 8. Mark this DC as printing, so **CScrollView::OnPrepareDC** won't modify the viewport:

```
pDC->m_bPrinting = TRUE;
```

☐ 9. Return the pointer to the CDC:

```
return pDC;
```

☐ 10. Save TextView.cpp.

Notes

To see an example of how your code should look, click this icon.
(CD-ROM displays the following sample code.)

```
CDC* CTextView::CreatePrinterDC()
{
    PRINTDLG PrtDlg;
    HDC      hDC;

    if (!AfxGetApp()->GetPrinterDeviceDefaults(&PrtDlg))
    {
        hDC = ::CreateDC("display",NULL,NULL,NULL);
    }
    else
    {
        CPrintDialog dlg(FALSE);
        dlg.m_pd.hDevMode = PrtDlg.hDevMode;
        dlg.m_pd.hDevNames = PrtDlg.hDevNames;
        hDC = dlg.CreatePrinterDC();
    }

    CDC* pDC = CDC::FromHandle(hDC);
    pDC->m_bPrinting = TRUE;
    return pDC;
}
```

▶ Implement OnBeginPrinting

In this procedure, you will invoke the **ComputePrintMetrics** function.

☐ 1. In the **CTextView::OnBeginPrinting**, make a call to **ComputePrintMetrics** that calculates the basic metrics for the printer DC:

```
ComputePrintMetrics();
```

☐ 2. Save TextView.cpp.

For more information about the **OnBeginPrinting** function, see Overriding CView-Derived Functions in this chapter.

▶ Implement OnPrint

In this procedure, you will override the function **OnPrint** to start actual printing.

☐ 1. Use ClassWizard or WizardBar to create **CTextView::OnPrint**.

☐ 2. Calculate which lines to print, using **ComputePrintableLines**:

```
int nFirstLn, nLastLn;
ComputePrintableLines(pDC, pInfo, nFirstLn, nLastLn);
```

☐ 3. Call the core **OnDraw** handler that you developed in Lab 4.3, Exercise 1: Implementing a Basic Text Viewer:

```
OnDraw(pDC, nFirstLn, nLastLn);
```

☐ 4. Save TextView.cpp.

Notes

To see an example of how your code should look, click this icon.

(CD-ROM displays the following sample code.)

```
void CTextView::OnPrint(CDC* pDC, CPrintInfo* pInfo)
{
    int nFirstLn, nLastLn;
    ComputePrintableLines(pDC, pInfo, nFirstLn, nLastLn);
    OnDraw(pDC, nFirstLn, nLastLn);
}
```

▶ **Implement the ComputePrintableLines function**

☐ 1. Declare a protected function in TextView.h as follows:

```
void ComputePrintableLines( CDC* pDC, CPrintInfo* pInfo,
                            int& nFirst, int& nLast);
```

☐ 2. Save TextView.h.

☐ 3. In TextView.cpp, define **ComputePrintableLines** as follows:

```
void CTextView::ComputePrintableLines( CDC*         pDC,
                                       CPrintInfo*  pInfo,
                                       int&         nFirst,
                                       int&         nLast)
```

☐ 4. Get the document so that you will know how many lines are available to print as follows:

```
CTextDoc* pDoc = GetDocument();
```

☐ 5. Set the first line equal to the number of lines per page multiplied by the zero-based page number:

```
nFirst = GetLinesPerPrintPage() * (pInfo->m_nCurPage-1);
```

☐ 6. Set the last line equal to the first line plus the number of lines per page (or the last line in the document) as follows:

```
nLast  = min( nFirst + GetLinesPerPrintPage() - 1,
              pDoc->GetLineList()->GetCount() - 1);
```

☐ 7. Save TextView.cpp.

To see an example of how your code should look, click this icon.

(CD-ROM displays the following sample code.)

```
void CTextView::ComputePrintableLines( CDC*         pDC,
                                       CPrintInfo*  pInfo,
                                       int&         nFirst,
                                       int&         nLast)
{

    CTextDoc* pDoc = GetDocument();

    nFirst = GetLinesPerPrintPage() * (pInfo->m_nCurPage-1);
    nLast  = min( nFirst + GetLinesPerPrintPage() - 1,
                  pDoc->GetLineList()->GetCount() - 1);
}
```

Notes

▶ **Build and test your application**

- Run your application. Open a file and try printing using the **Print** option. Use Print Preview to preview the file.

The solution code for this exercise is located in the folder \Labs\Ch10\Lab10.1\Ex01\Solution.

Notes